AI ASSISTANT OVERSIGHT AND RESPONSIBLE AUTOMATION

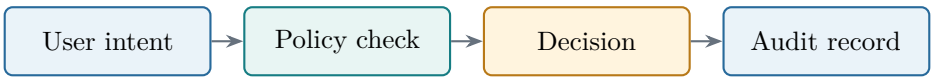
# Black Box for Alexa+

## A Practical Framework for Explainable Actions, Guardrails, and User Oversight

**Umar Tabbsum**

Independent Researcher and ML Software Engineer

An assistant should not only perform an action. It should make the action understandable, expose the rules behind the decision, and provide a meaningful record that a person can inspect.




**Project repository:** https://github.com/u19725505-star/black-box-alexa
**Interactive demonstration:** https://u19725505-star.github.io/black-box-alexa/
**MCP demonstration page:** https://u19725505-star.github.io/black-box-alexa/mcptest.html

Research and prototype description. The current implementation demonstrates simulated actions and oversight workflows; it is not a claim of direct integration with production Alexa devices.

**Abstract**

Voice assistants and AI agents are becoming capable of handling tasks that affect everyday life. As their responsibilities grow, the question is no longer simply whether an assistant can complete a request. It is also whether a person can understand what the assistant intended to do, why a request was permitted or held, and what record remains afterward.

This paper presents Black Box for Alexa+, a prototype for making assistant actions more inspectable and easier to govern. The project combines policy checks, configurable guardrails, human-readable explanations, a hash-linked event ledger, simulated undo operations, and a Model Context Protocol (MCP) server exposing nine tools. Demonstration scenarios include a routine grocery purchase, a purchase above a configured threshold, a thermostat request during quiet hours, and inspection of ledger integrity.

The central contribution is a practical interaction pattern: evaluate an action against explicit rules, explain the resulting decision, and preserve a record that a person can inspect. The prototype is intentionally presented as an oversight layer rather than a replacement for the assistant itself. It does not establish production-grade security, guarantee tamper-proof storage, or execute real-world Alexa, shopping, thermostat, or calendar actions. Those boundaries inform both the current implementation and the evaluation work required before deployment in a real environment.

**Keywords:** AI agents, voice assistants, Alexa, Model Context Protocol, human oversight, policy enforcement, auditability, explainable AI, responsible automation.

# 1 Introduction

A useful assistant should make everyday tasks easier. A trustworthy assistant must also help people understand what happens when a request carries a cost, changes a device setting, or conflicts with a rule the user previously established.

Imagine asking an assistant to order groceries. The request may be routine, and the cost may fall within the user's budget. Now imagine a second request for expensive headphones that exceeds a spending limit. A third request might involve changing a thermostat late at night, when a quiet-hours rule should apply. These situations are not identical, and a well-designed assistant should not treat them as though they were.

The challenge is not merely producing a yes-or-no response. People need to know which rule was applied, what information influenced the result, and whether they have an opportunity to review or reverse the outcome.

Black Box for Alexa+ explores that problem through a small, inspectable prototype. Rather than claiming that a single interface can solve the entire problem of AI safety, the project concentrates on a concrete engineering question:

> *How can an assistant's proposed actions become easier to inspect, explain, constrain, and audit before those actions are trusted with consequential tasks?*

The prototype demonstrates policy decisions, explanations, guardrail configuration, event recording, ledger verification, and simulated undo behavior. A browser-based interface presents these ideas in an approachable form, while an MCP server makes the capabilities available through structured tool calls.

The goal is practical rather than speculative. The project provides a foundation for testing oversight patterns, identifying bypass conditions, and deciding what additional controls would be needed before connecting such a system to real devices or services.

## 1.1 Problem statement

An assistant may understand a user's request and still lack a reliable mechanism for explaining or documenting its decision. Without a clearly defined control layer, a user may have difficulty answering questions such as:

- Was the requested action checked against a policy?
- Which rule caused the action to proceed or be held?
- Can the user inspect the relevant event afterward?
- Can an accidental or unwanted action be reversed?
- Can an administrator verify whether the audit history has been altered?

These questions become particularly important when automation involves money, household devices, personal schedules, or other activities where mistakes have real consequences.

### 1.2 Design objectives

The project is guided by five objectives.

1. **Make decisions visible.** Show whether an action is allowed, held, or blocked.
2. **Explain the result.** Translate a policy decision into language a person can understand without inspecting source code.
3. **Make rules configurable.** Provide explicit guardrails, including spending limits and quiet-hours settings.
4. **Preserve an inspectable history.** Link event records cryptographically so certain changes to the stored history can be detected.
5. **Support structured tool access.** Expose oversight functions through MCP tools that can be exercised independently of the visual interface.

## 2 Contributions

The contribution of Black Box for Alexa+ is the integration of several complementary oversight ideas into a working demonstration.

**Table 1:** *Primary project contributions and their intended value.*

| Contribution | What it provides | Practical value |
|---|---|---|
| Policy checks | Evaluates proposed actions against configured rules. | Makes decisions more predictable and inspectable. |
| Decision explanations | Provides a human-readable account of a decision. | Helps users understand why an action was held or allowed. |
| Guardrail configuration | Allows policy settings to be changed. | Makes the oversight behavior adaptable to different needs. |
| Hash-linked ledger | Connects event records through cryptographic hashes. | Can reveal certain unauthorized changes to recorded history. |
| Undo demonstration | Records a simulated reversal or undo operation. | Illustrates how corrective workflows could be represented. |
| MCP tool interface | Exposes nine named oversight tools. | Creates a structured interface for agent and developer testing. |

These contributions should be understood in the context of a prototype. They demonstrate an approach and provide a starting point for testing; they do not establish that every component is secure against a determined attacker or ready for production.

## 3 System Overview

The implementation has a browser-based demonstration and a server-side MCP interface. The browser presents scenarios and visual feedback. The server exposes structured operations for checking, recording, explaining, undoing, and verifying actions.

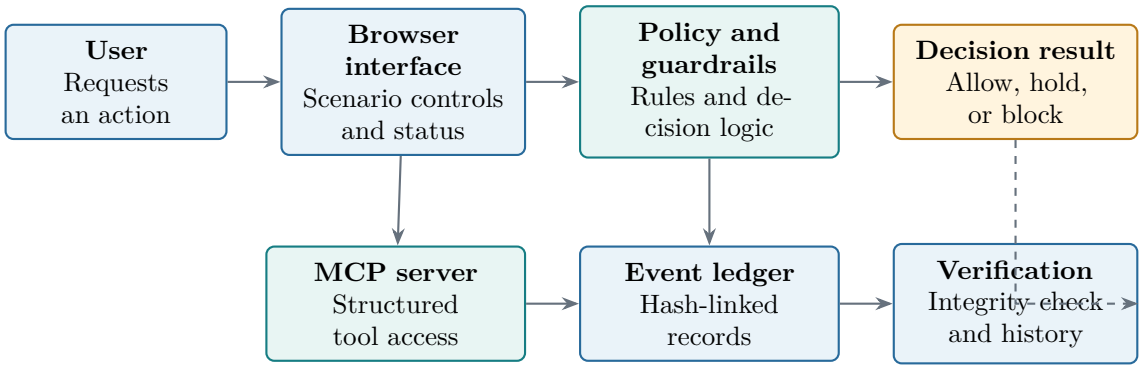



**Figure 1:** *Conceptual system architecture. The diagram represents the intended oversight workflow, not a claim that every action necessarily passes through a single shared production ledger. The current browser and server components maintain separate state in some workflows.*

### 3.1 Browser-based interface

The browser interface makes the prototype approachable for people who do not want to interact with an API directly. It presents simulated action scenarios, displays decisions, and provides controls for exploring explanations and ledger behavior.

The demonstration is useful because it makes the policy process visible. A reviewer can compare a routine request with a request that crosses a threshold, rather than relying on a description of what the system is supposed to do.

### 3.2 MCP server

The server exposes structured tools through the Model Context Protocol. MCP provides a standard way for compatible clients to discover and invoke tools exposed by a server. In this project, the tools cover action checking, recording, explanation, guardrail management, ledger inspection, integrity verification, and demonstration scenarios.

The MCP interface is a prototype integration point. It should not be interpreted as proof of a direct connection to a production Alexa service or as authorization to perform real-world actions.

### 3.3 State and storage boundaries

An important implementation detail is that the browser and server do not automatically constitute one shared, authoritative record for every operation. The browser maintains local application state, while the server uses in-memory ledger state. Server state can be lost on restart, and separate records can diverge.

A production design would need to establish one authoritative persistence layer and define how events are committed, synchronized, retried, and reconciled.

## 4 The Decision Workflow

The project organizes its demonstration around a simple sequence: receive a proposed action, evaluate its policy conditions, explain the result, and record the relevant event.

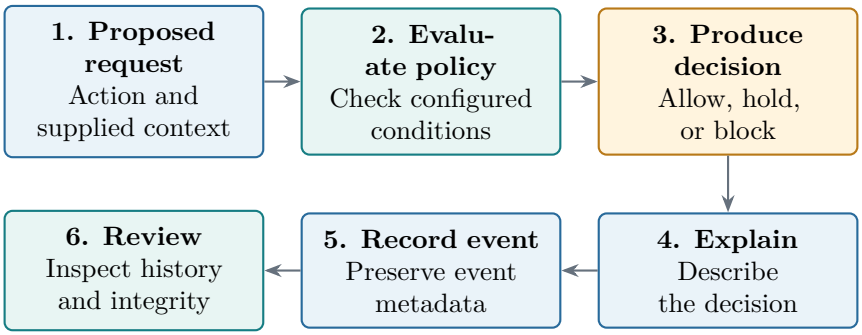



**Figure 2:** *The proposed oversight workflow. In a production implementation, policy enforcement, recording, and any downstream execution would need transactional and security guarantees.*

### 4.1 Three meaningful outcomes

A useful policy interface should distinguish between different types of decisions.

- **Allowed:** the proposed action satisfies the rules evaluated by the current implementation.
- **Held:** the action requires review or further authorization before it should proceed.
- **Blocked:** the action conflicts with a rule that prevents it from proceeding under the current policy.

The distinction matters because not every unusual action should be treated as prohibited. Some actions may be acceptable after confirmation; others should remain unavailable while a rule is in effect.

A production implementation must also ensure that these labels correspond to actual enforcement. Displaying a held status is not the same as preventing a downstream service from executing the action.

### 4.2 Illustrative policy logic

The following pseudocode describes a desired policy pattern. It is a conceptual example, not a verbatim listing of the current implementation.

**Listing 1:** *Illustrative policy evaluation*

```
function evaluateAction(action, policy, context):
    if action is invalid:
        return BLOCKED("Invalid action")

    if policy requires quiet_hours_check:
        if context.quiet_hours:
            return BLOCKED("Quiet-hours rule applies")

    if action.amount is present:
        if action.amount > policy.spending_limit:
            return HELD("Spending threshold exceeded")

    return ALLOWED("Configured checks passed")
```

For a real system, this logic would need strict schema validation, trusted context, authenticated authorization, and fail-closed behavior when required information is missing.

## 5 Demonstration Scenarios

The prototype uses familiar household examples to make the oversight problem concrete.

**Table 2:** *Demonstration scenarios and the behavior they illustrate.*

| Scenario | Illustrative condition | Oversight lesson |
|---|---|---|
| Grocery purchase | A simulated $18 purchase under a default $25 threshold. | Routine requests can be checked against an explicit policy. |
| Headphones purchase | A simulated $79 purchase above the default $25 threshold. | A threshold can trigger a hold instead of silently treating every request as routine. |
| Thermostat at night | A request is evaluated against a quiet-hours condition. | Time-dependent policies can affect the decision. |
| Calendar action | A simulated calendar-related request is represented in the demonstration. | The same oversight concept can apply beyond purchases. |
| Explain a decision | The user asks why a decision occurred. | A decision should have an understandable explanation. |
| Undo an action | The system demonstrates a reversal record or status change. | Corrective workflows should be visible and reviewable. |
| Tamper demonstration | A ledger record is deliberately altered in a test scenario. | Integrity verification can detect certain changes to history. |

These examples are simulated. They do not establish that the prototype actually purchases groceries, orders headphones, changes a physical thermostat, or modifies a live calendar.

### 5.1 Why the scenarios matter

The value of the scenarios is the contrast between requests. A routine request and a request that crosses a threshold should not receive identical treatment if the user's policy distinguishes between them. Similarly, a quiet-hours rule should be represented as a rule rather than hidden behind a vague response.

The demonstration makes these decisions inspectable. It also reveals a broader engineering requirement: the system must not allow an untrusted caller to rewrite the facts that determine whether a request is permitted.

## 6 MCP Tool Surface

The MCP server exposes nine tools. Together, they demonstrate a structured interface for checking and reviewing actions.

**Table 3:** *The nine MCP tools and their intended roles.*

| Tool | Purpose | Review question |
|---|---|---|
| `check_action` | Evaluates a proposed action against the current policy logic. | What decision does the policy return? |
| `record_action` | Adds an action event to the ledger. | What was recorded? |
| `explain_action` | Provides a human-readable explanation of an action or decision. | Why did this outcome occur? |
| `undo_action` | Demonstrates a reversal or undo operation in the record. | How is a correction represented? |
| `set_guardrail` | Changes a configurable policy setting. | Which rule can be adjusted? |

| Tool | Purpose | Review question |
|---|---|---|
| `verify_chain` | Checks the integrity of the hash-linked ledger. | Does the current chain verify? |
| `list_ledger` | Returns ledger entries for inspection. | What history is available? |
| `demo_tamper` | Runs a demonstration that alters ledger data. | Can a deliberate change be detected? |
| `demo_reset` | Resets demonstration state. | Can the test environment be restored? |

The tool names indicate the intended capabilities. Their presence does not, by itself, prove authorization, complete policy coverage, durable storage, or secure integration with an external device.

### 6.1 Why MCP is useful here

MCP gives the prototype a structured tool boundary rather than requiring every capability to exist only as a button in a webpage. That boundary makes the functions easier to test independently and creates a potential integration path for compatible clients.

However, a tool protocol is not a security policy by itself. Authentication, authorization, validation, rate limits, and operational controls must be implemented at the appropriate boundaries.

## 7 Ledger Design and Integrity Verification

The prototype uses a hash-linked ledger. Conceptually, each event contains information about the event and a hash that links it to the preceding entry.

Let $E_i$ denote the event data for entry $i$, let $H_i$ denote its stored hash, and let $H_{i-1}$ denote the previous entry's hash. A simplified construction is

$$H_i = \mathrm{SHA256}(E_i \,\|\, H_{i-1}),$$

where $\|$ denotes concatenation.

The first record uses a defined initial value or genesis reference. For verification, the implementation recomputes each expected hash and checks whether the stored values remain consistent.

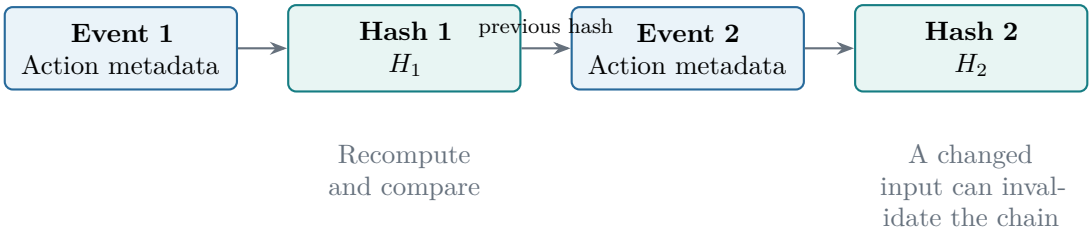



**Figure 3:** *Conceptual hash linkage. A chain can reveal certain changes to hashed fields, but it is not automatically immutable or tamper-proof.*

### 7.1 What hash linkage can provide

Hash linkage can make accidental edits or some forms of unauthorized modification detectable. If a stored field contributes to the hash and is changed without recomputing the relevant hashes, verification can fail.

This is useful because an audit record is more valuable when a reviewer can test whether the data still matches its expected cryptographic structure.

### 7.2 What it cannot guarantee

A hash chain alone does not prevent an attacker with sufficient access from rewriting entries and recomputing the chain. It also does not establish who created a record, whether the original event was truthful, or whether a record was omitted before it reached the ledger.

The current prototype has additional limitations:

- The server ledger is maintained in memory and can be lost when the process restarts.
- The browser and server may maintain separate histories.
- The undo status is not included in every ledger fingerprint used by the current implementation.
- The design does not establish an independent trusted checkpoint or externally anchored history.

A stronger design would use durable append-only storage, canonical serialization, authenticated event authorship, access controls, external checkpoints, and explicit verification of every security-relevant field.

## 8 Human Oversight and Explainability

An explanation is useful when it answers a practical question. A person should not need to infer the reason for a hold from a color or status label alone.

For example, a spending-related explanation might state that the proposed amount exceeds the configured threshold and therefore requires confirmation. A quiet-hours explanation might state that the requested change conflicts with the active time-based rule.

These explanations are most useful when they are:

- **Specific:** they identify the rule that influenced the decision.
- **Consistent:** the explanation agrees with the underlying decision logic.
- **Understandable:** the user does not need to read source code to interpret the result.
- **Reviewable:** the explanation can be associated with the relevant event.
- **Honest about uncertainty:** missing or untrusted information is not presented as verified fact.

The project demonstrates an explanation interface, but explanation quality requires systematic testing. A production system should verify that explanations are grounded in actual policy outcomes rather than generated after the fact without a reliable connection to the decision.

## 9 Threat Model and Security Considerations

A system that governs actions must consider not only ordinary mistakes but also attempts to bypass its rules.

The following threat model identifies relevant risks in the current prototype and the controls needed for a more robust implementation.

**Table 4:** *Threats, current concerns, and recommended mitigations.*

| Threat | Current concern | Recommended mitigation |
|---|---|---|
| Caller bypasses policy | A caller-supplied approval value can influence whether an action is held. | Enforce authorization on the server and verify approval using trusted identity and policy state. |
| Untrusted context | A caller can supply context such as whether it is nighttime. | Obtain sensitive policy inputs from trusted sources or explicitly treat them as unverified. |
| Invalid numeric input | Missing or malformed amounts may not be handled safely in every path. | Use strict schemas, reject invalid values, and fail closed when required fields are missing. |
| Unauthorized guardrail changes | The server does not provide sufficient application-level authorization for every sensitive operation. | Require authenticated principals and least-privilege permissions. |
| Public destructive tools | Reset and tamper demonstration tools can modify shared state. | Disable destructive demo tools in production or protect them behind explicit authorization. |
| Cross-origin access | Wildcard CORS may allow requests from arbitrary origins. | Use an origin allowlist where appropriate, plus authentication and server-side authorization. |
| Shared server state | Multiple clients can interact with module-level in-memory state. | Isolate users and sessions and persist records in a durable store. |
| Ledger rewriting | A privileged attacker may alter entries and recompute hashes. | Use external checkpoints, signed records, restricted writes, and independent verification. |
| Browser/server divergence | Separate event histories may disagree. | Define a canonical event service and use idempotent writes, reconciliation, and transaction boundaries. |
| Simulated undo | An undo record does not necessarily reverse an external action. | Implement provider-specific compensation and verify its result. |

## 9.1 The difference between a demonstration and enforcement

The most important security distinction is between reporting a decision and enforcing it.

If a tool returns “held” but an untrusted caller can bypass the decision or invoke a downstream action independently, the policy has not been reliably enforced. Likewise, a user interface that displays a guardrail does not establish that every request passes through that guardrail.

For deployment, the enforcement boundary should sit on the server-side path immediately before any consequential external operation. A client should not be able to grant itself permission by supplying a Boolean field or changing local state.

### 9.2 Security design principles

A stronger implementation should follow these principles:

1. Validate every incoming request against a strict schema.
2. Derive policy inputs from trusted sources whenever possible.
3. Authenticate the caller before sensitive operations.
4. Authorize each action independently of the user interface.
5. Fail closed when mandatory information is missing.
6. Separate demonstration controls from production controls.
7. Record policy versions and relevant decision metadata.
8. Protect audit records with durable storage and independent verification.
9. Treat undo as a compensating operation whose success must be confirmed, not as a label change.

## 10 Evaluation Plan

A credible evaluation should test more than whether the happy-path demo works. It should measure whether the policy behaves correctly under ordinary, boundary, malformed, and adversarial conditions.

No formal performance or security benchmark results are claimed in this paper. The following table defines a practical evaluation plan that can be completed and reported using reproducible tests.

**Table 5:** *Proposed evaluation matrix. Results must be populated from actual test runs.*

| Test category | Test procedure | Metric or outcome |
|---|---|---|
| Policy correctness | Test allowed, held, and blocked actions across defined cases. | Correct decision rate. |
| Threshold boundaries | Test amounts below, equal to, and above the configured limit. | Boundary-case pass rate. |
| Malformed inputs | Submit missing, negative, nonnumeric, and out-of-range values. | Invalid requests rejected. |
| Authorization | Attempt sensitive operations without appropriate permission. | Unauthorized operations denied. |
| Context integrity | Modify caller-supplied context to try to bypass a rule. | Bypass attempts prevented. |
| Ledger integrity | Change hashed fields and verify the chain. | Detection rate for tested modifications. |
| Ledger persistence | Restart the service and inspect prior events. | Events retained after restart. |
| Undo correctness | Test successful and failed compensating operations. | Verified reversal rate. |
| Concurrency | Send simultaneous requests that modify shared state. | Consistency and duplicate-event rate. |
| Usability | Ask representative users to explain a decision after seeing it. | Comprehension and task completion rates. |

### 10.1 Recommended reporting metrics

Let $N$ be the number of test cases and $C$ the number of cases where the system returns the expected decision. The observed policy correctness rate can be reported as

$$\text{Correctness} = \frac{C}{N} \times 100\%.$$

For a defined set of unauthorized test attempts, let $B$ denote the number successfully blocked and $U$ the total number of attempts. A simple blocking rate is

$$\text{Blocking Rate} = \frac{B}{U} \times 100\%.$$

These measures are meaningful only when the test cases, expected outcomes, and threat assumptions are documented. A high rate on a small test set would not, by itself, prove security.

For ledger testing, report the number and types of mutations attempted, the number detected, and the types that went undetected. A single successful tamper demonstration should not be generalized to all possible attacks.

## 11 Design Comparison

A conventional assistant interaction often focuses on whether the request was completed. The oversight pattern explored here adds decision transparency and a record of what happened.

**Table 6:** *Conceptual comparison of interaction patterns.*

| **Dimension** | **Basic interaction pattern** | **Oversight pattern demonstrated here** |
|---|---|---|
| Action decision | May be presented primarily as a final response. | Separates policy evaluation from the explanation of its result. |
| Rules | May be difficult to inspect from the interaction alone. | Makes configurable guardrails part of the demonstration. |
| Explanation | May provide a short status or conversational response. | Provides a dedicated explanation operation. |
| History | May not expose a unified audit trail to the user. | Demonstrates a hash-linked ledger and history inspection. |
| Correction | May depend on the external service's own features. | Demonstrates an undo-related record, while acknowledging that real compensation requires external integration. |
| Verification | May not include an explicit integrity-check operation. | Includes a tool for verifying the stored hash chain. |

This is a comparison of design emphasis, not an empirical claim that every existing assistant lacks these capabilities. Many production systems already use permissions, logs, confirmations, and other safety mechanisms. The project focuses on making those concepts visible together in a small demonstration.

## 12 Limitations

The current implementation has several limitations that matter for interpreting its contribution.

### 12.1 No direct production Alexa integration

The prototype is not a verified Alexa+ integration. It does not establish that the tools are registered with, or invoked by, production Alexa services. Its value is as an independent demonstration of a possible oversight layer.

### 12.2 Simulated actions

The household examples are simulated. The system does not prove that a purchase was sent to a retailer, that a thermostat changed, or that a calendar event was created or reversed.

### 12.3 In-memory server ledger

The server-side ledger is not a durable production database. State can be lost when the process restarts. A real audit service would require persistent storage, backup, access control, and recovery procedures.

### 12.4 Separate state boundaries

The browser and server can maintain distinct records. Without a canonical event service and reconciliation protocol, those records should not be treated as a single authoritative history.

### 12.5 Incomplete authorization controls

The prototype does not establish production-grade authentication or authorization for every sensitive operation. Caller-controlled inputs and exposed demonstration tools are important risks that must be addressed before deployment.

### 12.6 Limited empirical evaluation

This paper does not report measured latency, security-test coverage, user-study outcomes, or benchmark comparisons. Those results must come from documented experiments rather than assumptions about the design.

These limitations are not incidental. They identify the work needed to move from a convincing demonstration toward a dependable system.

## 13 Future Work

The next development phase should prioritize enforcement, persistence, and reproducible evaluation before adding more demonstration features.

1. **Create a canonical event service.** Make the server the authoritative source for decisions and records, with explicit handling for retries and conflicts.
2. **Add durable persistence.** Store events in a database and define backup, recovery, and retention procedures.
3. **Strengthen authorization.** Protect guardrail changes, ledger access, resets, and other sensitive operations with authenticated identities and least-privilege permissions.

4. **Validate all policy inputs.** Reject malformed values, missing mandatory fields, and untrusted context when a decision depends on it.
5. **Separate test tools from operational tools.** Ensure that tamper and reset demonstrations cannot be invoked against a production ledger.
6. **Strengthen integrity guarantees.** Use canonical event serialization, signed records, restricted writes, and independent checkpoints.
7. **Define real undo semantics.** For every supported external action, specify whether reversal is possible and verify its completion.
8. **Run a repeatable evaluation suite.** Publish test cases, expected outcomes, observed results, and known failures.
9. **Evaluate human comprehension.** Test whether users can accurately explain why a request was allowed, held, or blocked.

A measured, security-first development path would make the project more useful than adding integrations before the underlying decision and audit guarantees are established.

## 14 Conclusion

Black Box for Alexa+ explores a practical idea: an assistant should not be judged only by how quickly it responds, but also by whether people can understand and review its consequential decisions.

The prototype brings policy checks, explanations, configurable guardrails, a hash-linked ledger, simulated undo behavior, and nine MCP tools into one demonstration. Its household scenarios make the problem tangible, while its integrity checks illustrate how cryptographic linkage can help identify certain changes to stored records.

The project also exposes a distinction that matters for responsible AI engineering. A visible policy is not necessarily an enforced policy, and a hash-linked log is not necessarily an immutable audit system. Caller-controlled inputs, in-memory storage, separate browser and server state, and incomplete authorization remain issues to resolve.

The next step is therefore clear: preserve the usability of the demonstration while strengthening the enforcement boundary, persistence model, and test evidence. With those improvements, the approach could provide a useful foundation for making assistant-driven workflows more understandable and accountable.

The central principle is straightforward: when an assistant proposes a consequential action, the user should have a clear way to understand the decision, inspect the record, and know what protections are actually in place.

## 15 Judging Panel

The following table identifies the judging panel supplied for the project review. Names and roles are reproduced as provided for identification of the intended audience. Inclusion in this document does not imply endorsement, participation in the project, or a stated opinion about its quality.

**Table 7:** *Judges and professional roles.*

| Judge | Role |
|---|---|
| Jon Kirk | Vice President, Amazon Devices |
| Joshua Danovitz | Vice President, Fire TV Experience |
| Dawn Martynuik | Senior Manager, Fire TV Voice & AI |
| Macy Armstrong | Director, Fire TV Devices |
| Karthik Ragubathy | Senior Solutions Architect, Alexa |
| Emerson Sklar | Chief Evangelist, Alexa |
| Saylee Joshi | Senior Solutions Architect, Alexa |
| Maria de Lourdes Zollo | Co-founder & GM, Bee |
| Ethan Sutin | Co-founder, Head of Software & AI, Bee |
| Kourtney Meiss | Developer Evangelist |
| Alex Clausen | Senior Manager, Ring Appstore |
| Roman Alekseenkov | Director of Engineering, Ring |
| Harish Komaragiri | Principal Product Manager, Ring Developers |
| Brittney Hurst | Principal Technical Business Development Manager, AWS |
| Chris Melissinos | Principal Evangelist, AWS |
| Mike Grabowski | Founding partner, Vega OS (Callstack) |
| Mo Gorhom | Senior Software Development Engineer |
| Matt Engler | Senior Technical Product Manager, NBA |
| Chris Traganos | Developer Relations Leader |

### 15.1 Relevance to the judging audience

The project intersects with several areas represented by the panel: voice experiences, device ecosystems, AI tools, developer platforms, and software engineering. Its most relevant question for reviewers is whether the proposed oversight pattern provides a useful, testable way to make assistant actions more transparent and controllable.

The paper therefore emphasizes practical demonstration, explicit boundaries, security gaps, and a path toward measurable validation rather than claiming capabilities that have not been implemented.

## A Project Links and Reproducibility

The following links identify the repository, browser demonstration, MCP test page, and hosted MCP endpoint.

**Table 8:** *Project resources.*

| Resource | Address |
|---|---|
| Source repository | https://github.com/u19725505-star/black-box-alexa |
| Browser demonstration | https://u19725505-star.github.io/black-box-alexa/ |
| MCP test page | https://u19725505-star.github.io/black-box-alexa/mcptest.html |
| Hosted MCP endpoint | |

For a reproducible review, the reviewer should inspect the source, run the demonstration scenarios, test the

MCP tools, and compare the observed behavior with the limitations and security concerns described in this paper.

A reproducible evaluation should record the date of testing, software revision, environment, inputs, expected decisions, observed results, and any errors. The existence of a live endpoint does not guarantee that it is continuously available or that its state is persistent.

## B Suggested Reviewer Walkthrough

A reviewer can assess the core idea using the following sequence.

1. Open the browser demonstration and inspect the available household scenarios.
2. Run the routine grocery scenario and observe the decision and the associated explanation.
3. Run the higher-value headphone scenario and inspect how the configured spending threshold affects the result.
4. Explore the quiet-hours scenario and note which context influences the policy outcome.
5. Inspect the explanation and ledger tools to see how the prototype presents its decision history.
6. Run the integrity demonstration in the intended test environment and observe whether the verification tool detects the injected modification.
7. Open the MCP test page and inspect the available tools.
8. Review the server-side implementation for input validation, authorization, shared state, and the handling of destructive demonstration operations.

This walkthrough is a suggested review procedure, not a claim that every step has passed an independent security assessment.

## C Judge Profile References

The following profile-image addresses were supplied with the judging panel information. They are listed as references only; the images are not downloaded or embedded in this paper.

1. Jon Kirk: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/592/datas/large.png
2. Joshua Danovitz: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/639/datas/large.png
3. Dawn Martynuik: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/640/datas/large.png
4. Macy Armstrong: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/641/datas/large.png
5. Karthik Ragubathy: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/646/datas/large.png
6. Emerson Sklar: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/647/datas/large.png

7. Saylee Joshi: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/648/datas/large.png
8. Maria de Lourdes Zollo: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/649/datas/large.png
9. Ethan Sutin: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/650/datas/large.png
10. Kourtney Meiss: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/651/datas/large.png
11. Alex Clausen: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/652/datas/large.png
12. Roman Alekseenkov: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/657/datas/large.png
13. Harish Komaragiri: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/658/datas/large.png
14. Brittney Hurst: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/659/datas/large.png
15. Chris Melissinos: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/660/datas/large.png
16. Mike Grabowski: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/661/datas/large.png
17. Mo Gorhom: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/662/datas/large.png
18. Matt Engler: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/528/664/datas/large.png
19. Chris Traganos: https://d112y698adiu2z.cloudfront.net/photos/production/judge_photos/005/630/532/datas/large.png

## References

[1] Model Context Protocol. *Model Context Protocol Documentation*. Available at: https://modelcontextprotocol.io/

[2] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS PUB 180-4. Available at: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[3] OWASP Foundation. *OWASP API Security Project*. Available at: https://owasp.org/www-project-api-security/

[4] Constantine, J. *Black Box for Alexa+: Project Repository*. Available at: https://github.com/u19725505-star/black-box-alexa

[5] Constantine, J. *Black Box for Alexa+: Interactive Demonstration*. Available at: https://u19725505-star.github.io/black-box-alexa/